

Week 1: Deep Latent Variable Models (DLVMs)

Generative Models: The decoder $f_\theta : \mathcal{Z} \rightarrow \mathcal{H}$ transforms latent representations into parameters of the output density $\eta = f_\theta(\mathbf{z})$. Output densities depend on data type (e.g., Categorical for images, Gaussian for continuous data).

PPCA: The model is $p(\mathbf{x}|\mathbf{z}) = \mathcal{N}(\mathbf{x}|\mathbf{W}\mathbf{z} + \mathbf{b}, \sigma^2\mathbf{I})$. The MLE of $\mathbf{b}$ is the sample mean $\hat{\mathbf{b}} = \bar{\mathbf{x}}$. This follows from setting $\partial\ell/\partial\mathbf{b} = 0$, which gives $N\mathbf{C}^{-1}(\mathbf{b} - \bar{\mathbf{x}}) = \mathbf{0}$, and since $\mathbf{C}^{-1}$ is full rank, $\mathbf{b} = \bar{\mathbf{x}}$.

VAEs & ELBO: Models are learned via amortized variational inference by optimizing the Evidence Lower Bound:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{\mathbf{z} \sim q_\phi(\mathbf{z}|\mathbf{x})} [\log p_\theta(\mathbf{x}|\mathbf{z})] - KL(q_\phi(\mathbf{z}|\mathbf{x}) || p(\mathbf{z}))$$

The second term equals $\mathbb{E}_{q_\phi} [\log q_\phi(\mathbf{z}|\mathbf{x}) - \log p(\mathbf{z})]$ by the definition of KL divergence.

KL Divergence (Closed Form): For two univariate Gaussians:

$$KL[\mathcal{N}(\mu_1, \sigma_1^2) || \mathcal{N}(\mu_2, \sigma_2^2)] = \ln \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

Hierarchical VAE ELBO: For a two-level model with $Q(\mathbf{z}_1, \mathbf{z}_2|\mathbf{x}) = q(\mathbf{z}_2|\mathbf{z}_1)q(\mathbf{z}_1|\mathbf{x})$, applying Jensen's inequality and factoring the joint:

$$\text{ELBO}(\mathbf{x}) = \mathbb{E}_Q \left[\log p(\mathbf{x}|\mathbf{z}_1) - \ln \frac{q(\mathbf{z}_1|\mathbf{x})}{p(\mathbf{z}_1|\mathbf{z}_2)} - KL[q(\mathbf{z}_2|\mathbf{z}_1) || p(\mathbf{z}_2)] \right]$$

Priors & Posterior Collapse: The standard Gaussian prior can cause posterior collapse. Better priors include Mixture of Gaussians (MoG) or VampPrior, a mixture of approximate posteriors:

$$p_\lambda(\mathbf{z}) = \frac{1}{K} \sum_{k=1}^K q_\phi(\mathbf{z}|\mathbf{u}_k)$$

Shared decoders in hierarchical VAEs also help prevent posterior collapse.

Week 2: Normalizing Flows

Concept: A sequence of invertible, differentiable transformations (diffeomorphisms) maps from a base distribution such as $\mathcal{N}(0, \mathbf{I})$:

$$\mathbf{x} = T(\mathbf{u}) = T_K \circ \dots \circ T_1(\mathbf{u})$$

The inverse reverses the order: $(T_K \circ \dots \circ T_1)^{-1} = T_1^{-1} \circ \dots \circ T_K^{-1}$.

Volume Preservation: A transformation f is volume-preserving if $|\det \mathbf{J}_f| = 1$. Example: $f(\mathbf{x}) = -\mathbf{x} + \mathbf{a}$ has $\mathbf{J}_f = -\mathbf{I}$, so $|\det \mathbf{J}_f| = 1$.

Jacobian Chain Rule: For $h = g \circ f$, we have $\mathbf{J}_h = \mathbf{J}_g \mathbf{J}_f$, and therefore:

$$|\det \mathbf{J}_h| = |\det \mathbf{J}_g| |\det \mathbf{J}_f|$$

Exact Likelihood: Density is computed using the change of variables formula:

$$p_x(\mathbf{x}) = p_u(\mathbf{u}) |\det J_T(\mathbf{u})|^{-1}$$

For a composition of K flows, $\log p_x(\mathbf{x}) = \log p_u(\mathbf{u}) - \sum_{k=1}^K \log |\det \mathbf{J}_{T_k}|$.

Dequantization: To apply continuous flows to discrete data, uniform noise is added: $\mathbf{x}' = \mathbf{x} + \mathbf{u}$, where $\mathbf{u} \sim \mathcal{U}(-0.5, 0.5)$.

Masked Coupling Layer (RealNVP): Uses binary mask $\mathbf{b} \in \{0, 1\}^D$. Forward:

$$\mathbf{z}' = \mathbf{b} \odot \mathbf{z} + (1 - \mathbf{b}) \odot (\mathbf{z} \odot \exp(s(\mathbf{b} \odot \mathbf{z})) + t(\mathbf{b} \odot \mathbf{z}))$$

Inverse: $\mathbf{z} = \mathbf{b} \odot \mathbf{z}' + (1 - \mathbf{b}) \odot ((\mathbf{z}' - t(\mathbf{b} \odot \mathbf{z}')) \odot \exp(-s(\mathbf{b} \odot \mathbf{z}')))$. Log-determinant: $\sum_d (1 - b_d) s_d(\mathbf{b} \odot \mathbf{z})$. The Jacobian is triangular, so $\det J = \exp(\sum s)$. RealNVP uses checkerboard and channel masking.

Glow: Extends RealNVP by incorporating invertible 1×1 convolutions as a generalization of permutation layers. Flows can also serve as highly expressive priors for VAEs.

Week 3: Diffusion Models

Forward Process: A Markov chain gradually corrupts data into noise:

$$q(\mathbf{z}_t | \mathbf{z}_{t-1}) = \mathcal{N}(\mathbf{z}_t; \sqrt{1 - \beta_t} \mathbf{z}_{t-1}, \beta_t \mathbf{I})$$

The marginal is $q(\mathbf{z}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{z}_t; \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$, where $\bar{\alpha}_t = \prod_{s=1}^t (1 - \beta_s)$.

Convergence to Pure Noise: For any schedule $0 < \beta_1 < \dots < \beta_T < 1$:

$$\bar{\alpha}_T < (1 - \beta_1)^T \rightarrow 0 \text{ as } T \rightarrow \infty \implies q(\mathbf{z}_T | \mathbf{x}) \rightarrow \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Reverse Process: A neural network (often a U-Net) approximates the denoising step:

$$p(\mathbf{z}_{t-1} | \mathbf{z}_t) = \mathcal{N}(\mathbf{z}_{t-1}; \boldsymbol{\mu}_\theta, \boldsymbol{\Sigma}_\theta)$$

ELBO Decomposition: The DDPM ELBO decomposes as:

$$L = KL[q(\mathbf{z}_T | \mathbf{x}) || p(\mathbf{z}_T)] - \log p(\mathbf{x} | \mathbf{z}_1) + \sum_{t=2}^T KL[q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x}) || p(\mathbf{z}_{t-1} | \mathbf{z}_t)]$$

L_{t-1} **Simplification:** Each KL term reduces to a mean-matching objective. Since both $q(\mathbf{z}_{t-1} | \mathbf{z}_t, \mathbf{x})$ and $p(\mathbf{z}_{t-1} | \mathbf{z}_t)$ are Gaussians with isotropic covariance:

$$L_{t-1} = \frac{1}{2\sigma_t^2} \|\tilde{\boldsymbol{\mu}}(\mathbf{z}_t, \mathbf{x}) - \boldsymbol{\mu}_\theta(\mathbf{z}_t, t)\|^2 + C$$

where C does not depend on θ .

DDPM Loss: The ELBO further simplifies to predicting the noise ϵ via MSE:

$$\mathcal{L}_{\text{simple}} = \mathbb{E}_{\epsilon, t} [\|\epsilon - \epsilon_{\theta}(\mathbf{z}_t(\mathbf{x}_0, \epsilon), t)\|^2]$$

Continuous SDEs: Modeled as $d\mathbf{x} = f(\mathbf{x}, t) dt + g(t) d\mathbf{w}$. Reversing requires the score $\nabla_{\mathbf{x}} \log p_t(\mathbf{x})$, learned via denoising score matching. Variance Preserving (VP) SDEs use a linear noise schedule $\beta(t)$.

Latent Diffusion: Computations scale better by first projecting data into a low-dimensional VAE latent space before running the diffusion process.

Week 5: Representation Invariance

Rotation Invariance of PPCA: If $\hat{\mathbf{x}} = \mathbf{R}\mathbf{x}$ for rotation matrix $\mathbf{R}$ (i.e. $\mathbf{R}^\top \mathbf{R} = \mathbf{I}$), then:

$$\|\hat{\mathbf{x}}_i - \hat{\mathbf{x}}_j\| = \|\mathbf{R}(\mathbf{x}_i - \mathbf{x}_j)\| = \sqrt{(\mathbf{x}_i - \mathbf{x}_j)^\top \mathbf{R}^\top \mathbf{R}(\mathbf{x}_i - \mathbf{x}_j)} = \|\mathbf{x}_i - \mathbf{x}_j\|$$

Angles are also preserved since $\langle \mathbf{R}\mathbf{u}, \mathbf{R}\mathbf{v} \rangle = \mathbf{u}^\top \mathbf{R}^\top \mathbf{R} \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle$.

Immersed vs. Embedded Manifolds: A smooth map $f : \mathbb{R}^d \rightarrow \mathbb{R}^D$ spans an *immersed* manifold if $\mathbf{J}_f(\mathbf{z})$ is full rank for all $\mathbf{z}$ (i.e. columns of $\mathbf{J}$ are linearly independent). It spans an *embedded* manifold if additionally f is injective (has a left-inverse), ensuring no self-intersections.

Curve Length: For a curve $c : [0, 1] \rightarrow \mathbb{R}^D$, the Euclidean length is:

$$\text{length}(c) = \int_0^1 \|\dot{c}(t)\| dt$$

The speed function is $\|\dot{c}(t)\|$. Numerically, approximate by $\sum_i \|c(t_{i+1}) - c(t_i)\|$.

Identifiability: Too much flexibility in neural network decoders means latent representations can be reparameterized without changing the model fit, leading to unstable or unidentifiable representations.

Noisy Manifolds: Observations are modeled as an immersed manifold plus noise:

$$\mathbf{x} = \mathbf{W}f(\mathbf{z}) + \epsilon$$

where $\mathbf{W}$ is interpreted as a random projection.

Ensembles: Capturing model uncertainty (e.g., using multiple randomly initialized decoders) is vital for understanding topology. Well-calibrated uncertainty builds a “wall” around the data so that geodesic curves naturally follow the data’s topological structure.

Week 6: Riemannian Metrics & Geodesics

Riemannian Geometry: A deterministic decoder $f : \mathbb{R}^d \rightarrow \mathbb{R}^D$ immerses a manifold in observation space. The pull-back Riemannian metric is:

$$\mathbf{G}_z = \mathbf{J}_f(\mathbf{z})^\top \mathbf{J}_f(\mathbf{z})$$

Local norms: $\|\mathbf{v}\|_{\mathbf{G}_z} = \sqrt{\mathbf{v}^\top \mathbf{G}_z \mathbf{v}}$. For $\mathbf{G}_z = (1 + \|\mathbf{z}\|^2)\mathbf{I}$, angles are Euclidean (the scalar factor cancels), but norms depend on position.

Curve Energy & Geodesics: The energy of a curve γ is:

$$E[\gamma] = \int_0^1 \dot{\gamma}(t)^\top \mathbf{G}_{\gamma(t)} \dot{\gamma}(t) dt$$

Shortest paths (geodesics) minimize this energy. For the Euclidean metric $\mathbf{G} = \mathbf{I}$, geodesics are straight lines $c_t = (1 - t)\mathbf{x}_1 + t\mathbf{x}_2$.

Christoffel Symbols & Geodesic ODE: The connection coefficients are:

$$\Gamma_j^{ik}(c_t) = \frac{1}{2} [\mathbf{G}_{c_t}^{-1}]_{ij} \left(2 \frac{\partial [G_{c_t}]_{ij}}{\partial [c_t]_k} - \frac{\partial [G_{c_t}]_{jk}}{\partial [c_t]_i} \right)$$

Constant-speed geodesics satisfy the 2nd-order ODE:

$$[\ddot{c}_t]_j = - \sum_{i,k} \Gamma_j^{ik}(c_t) [\dot{c}_t]_k [\dot{c}_t]_i$$

For $\mathbf{G} = \mathbf{I}$ all Christoffel symbols are zero, confirming $\ddot{c}_t = \mathbf{0}$ (straight lines).

Density-Based Metric: A useful metric for data-aware geodesics is:

$$\mathbf{G}_x = \frac{1}{p(\mathbf{x}) + \epsilon} \mathbf{I}, \quad p(\mathbf{x}) = \frac{1}{N} \sum_{n=1}^N \mathcal{N}(\mathbf{x} | \mathbf{x}_n, \sigma^2 \mathbf{I})$$

This penalizes paths leaving the data support, forcing geodesics to follow the data manifold.

Week 7: Information Geometry & Parameter Manifolds

Information Geometry: Treats the decoder as mapping to the space of conditional distributions $p(\mathbf{x}|\mathbf{z})$ rather than directly to observation space. Distances are measured using the KL divergence, which is invariant to parameterization choices. In contrast, Euclidean distances in parameter space are not invariant—different parameterizations of the same distributions yield different distances.

Randomly Projected Manifold: For a stochastic decoder $h(\mathbf{x}) = (f(\mathbf{x}), \sigma(\mathbf{x}))^\top$ mapping to $\mathbb{R}^{2D}$, the pullback metric combines both mean and variance Jacobians:

$$\mathbf{G}_h = \mathbf{J}_f^\top \mathbf{J}_f + \mathbf{J}_\sigma^\top \mathbf{J}_\sigma$$

Parametrization Non-Invariance: For $\mathcal{N}_1 = \mathcal{N}(0, 1)$ and $\mathcal{N}_2 = \mathcal{N}(1, 4)$, the Euclidean distance depends on the chosen parametrization: $\|(\mu, \sigma)\| = \sqrt{2}$, $\|(\mu, \sigma^2)\| = \sqrt{10}$, etc. The KL-based (Fisher information) metric is parametrization-invariant.

ReLU Network Non-Identifiability: For the network $f(\mathbf{x}) = \theta_2 \text{ReLU}(\theta_1 \mathbf{x})$, rescaling $\theta_1 \rightarrow c\theta_1$, $\theta_2 \rightarrow \theta_2/c$ leaves the function unchanged. Two parameter vectors produce the same function if $\sum_l \log \theta_l = \sum_l \log \theta'_l$.

Parameter Manifolds: The metric on parameter space is the finite-sample Fisher information:

$$\bar{\mathbf{G}}_\theta = \frac{1}{N} \sum_n \mathbf{J}_\theta(\mathbf{x}_n)^\top \mathbf{J}_\theta(\mathbf{x}_n)$$

Geodesics in this space connect functionally equivalent parameters, allowing the model to interpolate data perfectly.

Week 9: Graphs & Node Embeddings

Graph Basics: A simple graph is $G = (V, E)$. Laplacians:

$$\text{Unnormalized: } \mathbf{L} = \mathbf{D} - \mathbf{A}, \quad \text{Random Walk: } \mathbf{L}_{rw} = \mathbf{I} - \mathbf{D}^{-1} \mathbf{A}$$

The Weisfeiler-Lehman test checks graph isomorphism by iteratively hashing the multi-set of neighborhood labels.

Node-Level Statistics:

- **Eigenvector centrality:** The centrality scores are the components of the principal eigenvector of $\mathbf{A}$ (corresponding to the largest eigenvalue). Centralities must be positive, so flip the sign if necessary.
- **Clustering coefficient:** $c_u = \frac{|\{(v,w) \in E: v,w \in \mathcal{N}(u)\}|}{\binom{\deg(u)}{2}}$, the fraction of neighbor pairs that are connected.

Random Walks: Starting from distribution $\mathbf{p}$, after one step the distribution is:

$$\mathbf{A} \mathbf{D}^{-1} \mathbf{p} = \mathbf{P} \mathbf{p}, \quad \mathbf{P} = \mathbf{A} \mathbf{D}^{-1}$$

The number of distinct paths of length t from node u is the sum of the u -th column of $\mathbf{A}^t$.

Shallow Embeddings: The encoder maps nodes to vectors (e.g., $\mathbf{z}_u = \mathbf{s}_u^\top \mathbf{Z}$). A sigmoid decoder $\text{DEC}(\mathbf{z}_u, \mathbf{z}_v) = \sigma(\mathbf{z}_u^\top \mathbf{z}_v + b)$ gives the probability of an edge. Note: $1 - \sigma(x) = \sigma(-x)$. Cross-entropy loss for a single edge observation $S_{u,v} \in \{0, 1\}$:

$$L_{u,v} = -(S_{u,v} \log P_{u,v} + (1 - S_{u,v}) \log(1 - P_{u,v}))$$

Losses: Optimization also minimizes MSE ($\|\mathbf{S} - \mathbf{Z} \mathbf{Z}^\top\|_F^2$) or uses random walks with negative sampling.

Week 10: Graph Neural Networks

Permutation Invariance: Aggregation functions must be permutation invariant. Sum, product, and maximum are permutation invariant; concatenation is not.

Message Passing: State updates are order-invariant:

$$\mathbf{h}_u^{(k+1)} = \text{update}(\mathbf{h}_u^{(k)}, \text{agg}(\{\mathbf{h}_v^{(k)} \mid v \in \mathcal{N}(u)\}))$$

Power Iteration GNN: A GNN with aggregate = $\sum_{v \in \mathcal{N}(u)} h_v^{(k)}$ and update = $m_{\mathcal{N}(u)}^{(k)} / \|\mathbf{A}\mathbf{h}^{(k)}\|$ implements power iteration:

$$\mathbf{h}^{(k+1)} = \frac{\mathbf{A}\mathbf{h}^{(k)}}{\|\mathbf{A}\mathbf{h}^{(k)}\|}$$

This converges (under mild conditions) to the dominant eigenvector of $\mathbf{A}$, i.e., it computes **eigenvector centrality**.

Aggregation: Common approaches include set pooling, Janossy pooling (averaging over randomized permutations), and Transformer-style attention:

$$\alpha_{u,v} \propto \exp(\mathbf{h}_u^\top \mathbf{W} \mathbf{h}_v)$$

Updates & Parameter Counting: Concatenation, residual skip-connections (adding the update to the previous state), or GRU-based gating help combat over-smoothing. For a basic GNN with state dimension D , the parameters per round are $\mathbf{W}_{\text{self}} \in \mathbb{R}^{D \times D}$, $\mathbf{W}_{\text{neigh}} \in \mathbb{R}^{D \times D}$, $\mathbf{b} \in \mathbb{R}^D$: total $(2D^2 + D) \times K$ parameters for K rounds (independent of graph size).

Geometric GNNs: Physical symmetries (translation and rotation) are handled via invariant/equivariant operations on separate scalar and vector features.

Week 11: Graph Theory & Generative Graph Models

Discrete Convolution & Shift Operator: A discrete-time convolution is $y[n] = \sum_k x[k]h[n-k]$. The unit delay operator D shifts a signal: $Dx[n] = x[n-1]$. A causal filter can then be written as $y[n] = (\sum_{k=0}^{\infty} h[k]D^k)x[n]$.

Graph Shift Operator: On a graph, the signal at each node spreads to neighbors at each step:

$$D\mathbf{x}[n] = \mathbf{A}\mathbf{x}[n-1]$$

Applying a finite filter $h[k]$ (nonzero for $0 \leq k \leq N$) to a constant signal $\mathbf{x}$ gives the graph convolution:

$$\mathbf{y} = \sum_{k=0}^N h[k] \mathbf{A}^k \mathbf{x}$$

Graph Fourier Transform (GFT): For the cycle graph with N nodes, the vector $\mathbf{u}_k$ with components $(\mathbf{u}_k)_n = e^{-i\frac{2\pi k}{N}n}$ is an eigenvector of $\mathbf{A}$ with eigenvalue $\lambda_k = 2\cos(2\pi k/N)$. Thus the DFT basis equals the graph eigenbasis. For any graph with eigendecomposition $\mathbf{A} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^*$, the GFT is $\tilde{\mathbf{x}} = \mathbf{U}^*\mathbf{x}$ and the inverse is $\mathbf{x} = \mathbf{U}\tilde{\mathbf{x}}$.

Spectral Convolution: In the spectral domain, the graph convolution becomes:

$$\tilde{\mathbf{y}} = \sum_{k=0}^N h[k] \mathbf{\Lambda}^k \tilde{\mathbf{x}}, \quad \text{equivalently} \quad \mathbf{y} = \mathbf{U} \left(\sum_{k=0}^N h[k] \mathbf{\Lambda}^k \right) \mathbf{U}^* \mathbf{x}$$

Both vertex-domain and spectral-domain approaches give identical results. Vertex-domain requires N matrix-vector products $O(|V|^2)$; spectral-domain requires one eigendecomposition $O(|V|^3)$ that can be precomputed.

Graphical Models: Mean-field variational inference fixed-point equations are analytically similar to GNN message passing updates.

Generative Graph Models: Classical approaches include Erdős-Rényi, Stochastic Block-models, and Preferential Attachment. VAEs for graphs use a probabilistic encoder:

$$\mathbf{Z} \sim \mathcal{N}(\mu(G), \Sigma(G))$$

to build node-level or graph-level latent representations. Performance is evaluated by comparing distributions of graph statistics (degree, clustering coefficient) using the Total Variation (TV) distance.